

# GNR 638: Assignment 2

## Pre-trained CNN Backbone Analysis for Aerial Image Classification

---

**Afnan Abdul Gafoor**

Roll No: 22B2505

Group 58

---

Indian Institute of Technology Bombay

March 2026

### **Abstract**

This report presents a systematic analysis of three pre-trained CNN backbones (**ResNet50**, **DenseNet121**, and **EfficientNet-B0**) on the Aerial Image Dataset (AID) for 30-class land-use classification. We conduct controlled experiments across five scenarios: linear probe transfer, fine-tuning strategies, few-shot learning, corruption robustness, and layer-wise feature probing. Our analysis reveals that DenseNet121 consistently outperforms the other architectures in transfer learning, data efficiency, and corruption robustness, while all three models achieve  $\sim 97\%$  accuracy under full fine-tuning. We provide detailed analysis of representation quality across network depths, gradient dynamics under different training regimes, and robustness under distribution shifts.

# Contents

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| <b>1</b>  | <b>Experimental Setup</b>                               | <b>2</b>  |
| 1.1       | Dataset . . . . .                                       | 2         |
| 1.2       | Model Selection . . . . .                               | 2         |
| 1.3       | Training Configuration . . . . .                        | 2         |
| 1.4       | Reproducibility . . . . .                               | 2         |
| <b>2</b>  | <b>Scenario 1: Linear Probe Transfer</b>                | <b>2</b>  |
| 2.1       | Results . . . . .                                       | 3         |
| 2.2       | Confusion Matrices . . . . .                            | 4         |
| 2.3       | Feature Embedding Visualization . . . . .               | 4         |
| 2.4       | Analysis . . . . .                                      | 5         |
| <b>3</b>  | <b>Scenario 2: Fine-Tuning Strategies</b>               | <b>6</b>  |
| 3.1       | Results . . . . .                                       | 6         |
| 3.2       | Accuracy vs Percentage of Unfrozen Parameters . . . . . | 7         |
| 3.3       | Convergence Comparison . . . . .                        | 7         |
| 3.4       | Gradient Norm Analysis . . . . .                        | 7         |
| 3.5       | Analysis . . . . .                                      | 7         |
| <b>4</b>  | <b>Scenario 3: Few-Shot Learning Analysis</b>           | <b>9</b>  |
| 4.1       | Results . . . . .                                       | 9         |
| 4.2       | Training Curves Under Different Data Regimes . . . . .  | 9         |
| 4.3       | Analysis . . . . .                                      | 9         |
| <b>5</b>  | <b>Scenario 4: Corruption Robustness Evaluation</b>     | <b>10</b> |
| 5.1       | Results . . . . .                                       | 10        |
| 5.2       | Corruption Visualizations . . . . .                     | 11        |
| 5.3       | Analysis . . . . .                                      | 11        |
| <b>6</b>  | <b>Scenario 5: Layer-Wise Feature Probing</b>           | <b>12</b> |
| 6.1       | Layer Selection . . . . .                               | 12        |
| 6.2       | Results . . . . .                                       | 13        |
| 6.3       | Accuracy vs Depth & Feature Norms . . . . .             | 13        |
| 6.4       | PCA Across Layers . . . . .                             | 14        |
| 6.5       | Analysis . . . . .                                      | 14        |
| <b>7</b>  | <b>Unexpected Finding</b>                               | <b>15</b> |
| <b>8</b>  | <b>Failure Case</b>                                     | <b>15</b> |
| <b>9</b>  | <b>Computational Budget</b>                             | <b>16</b> |
| <b>10</b> | <b>Conclusion</b>                                       | <b>16</b> |
| 10.1      | Key Findings . . . . .                                  | 16        |
| 10.2      | Answers to Intellectual Expectations . . . . .          | 17        |

# 1 Experimental Setup

---

## 1.1 Dataset

The **Aerial Image Dataset (AID)** contains approximately 10,000 aerial images organized into 30 land-use categories, including Airport, Beach, Bridge, Commercial, Desert, Farmland, Forest, and others. Images are in RGB format with varying resolutions. We applied a stratified 80/20 train-validation split (seed=42), resulting in **5,594 training** and **1,399 validation** samples.

**Data augmentation** (training only): resize to  $224 \times 224$ , random horizontal flip, random rotation ( $\pm 15^\circ$ ), and color jitter (brightness, contrast, saturation = 0.2). Validation uses only resize and ImageNet normalization.

## 1.2 Model Selection

We selected three architectures that represent diverse CNN design philosophies:

Table 1: Selected pre-trained models and their characteristics.

| Model           | Design Philosophy    | Total Params | MACs    | FLOPs   |
|-----------------|----------------------|--------------|---------|---------|
| ResNet50        | Residual connections | 23.57M       | 4.13G   | 8.26G   |
| DenseNet121     | Dense feature reuse  | 6.90M        | 2.83G   | 5.67G   |
| EfficientNet-B0 | Compound scaling     | 4.00M        | 384.64M | 769.27M |

All models use ImageNet pre-trained weights from the `timm` library. The final classification layers are replaced with a linear layer mapping to 30 AID classes.

## 1.3 Training Configuration

- **Optimizer:** Adam ( $\text{lr} = 1 \times 10^{-3}$  for linear probes,  $1 \times 10^{-4}$  for fine-tuning)
- **Loss:** Cross-Entropy
- **Batch size:** 32
- **Epochs:** 30 (full-data), 20 (few-shot)
- **Seed:** 42 (for reproducibility across all experiments)
- **Hardware:** NVIDIA P100 GPU (Kaggle), total compute  $\sim 8\text{h } 51\text{m}$

## 1.4 Reproducibility

All experiments use a fixed random seed (42) applied to Python, NumPy, PyTorch, and CUDA. Stratified splitting ensures consistent class distributions across all data subsets. The complete codebase is available at: <https://github.com/Afnnnan/GNR638/tree/main/Assignment2>.

# 2 Scenario 1: Linear Probe Transfer

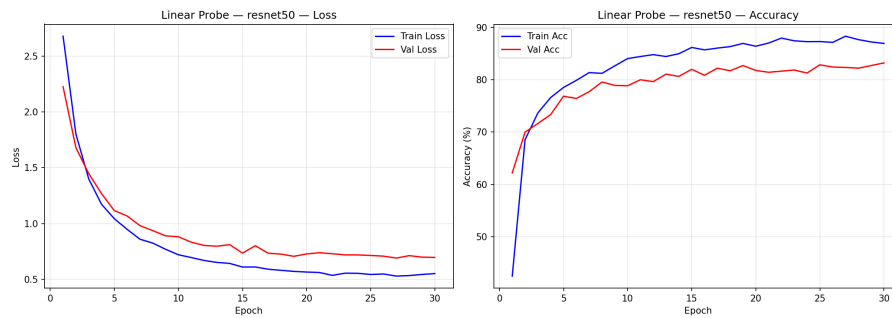
---

In this scenario, all backbone parameters are frozen and only a newly initialized linear classifier is trained. This evaluates the *intrinsic transferability* of ImageNet-learned representations to the AID aerial domain.

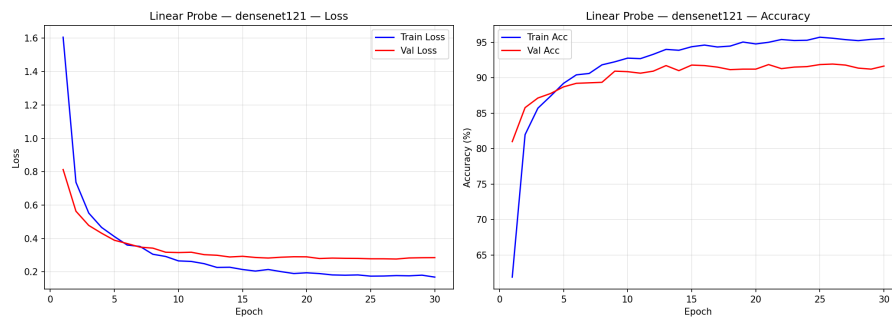
## 2.1 Results

Table 2: Linear probe results. Only the classifier head is trainable.

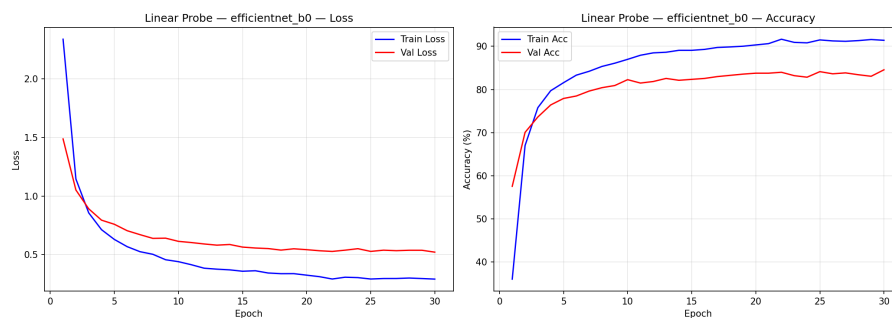
| Model           | Trainable Params | % Trainable | FLOPs   | Best Val Acc  |
|-----------------|------------------|-------------|---------|---------------|
| ResNet50        | 61,470           | 0.3%        | 8.26G   | 83.20%        |
| DenseNet121     | 30,750           | 0.4%        | 5.67G   | <b>91.92%</b> |
| EfficientNet-B0 | 38,430           | 0.9%        | 769.27M | 84.56%        |



(a) ResNet50



(b) DenseNet121



(c) EfficientNet-B0

Figure 1: Training and validation accuracy curves for linear probe transfer (30 epochs).

## 2.2 Confusion Matrices

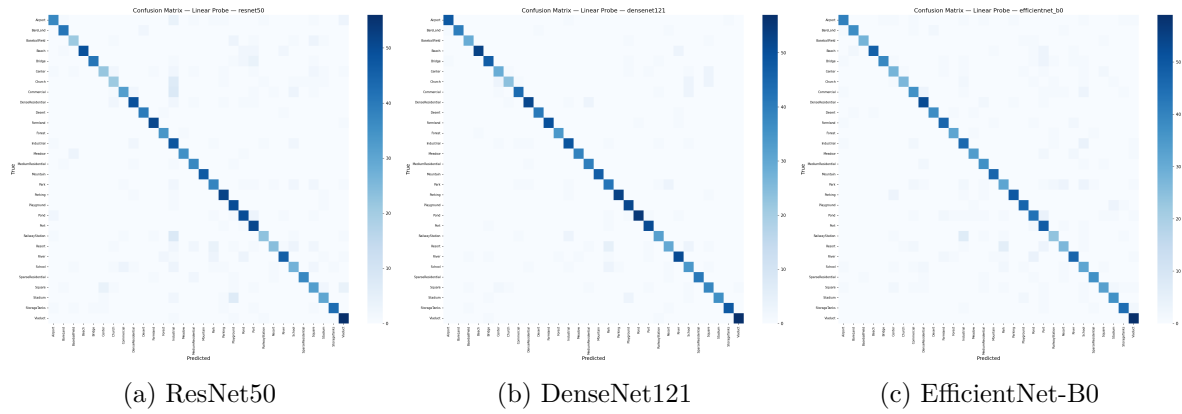


Figure 2: Confusion matrices for linear probe transfer. DenseNet121 shows the strongest diagonal, indicating superior class discrimination.

## 2.3 Feature Embedding Visualization

t-SNE projections:

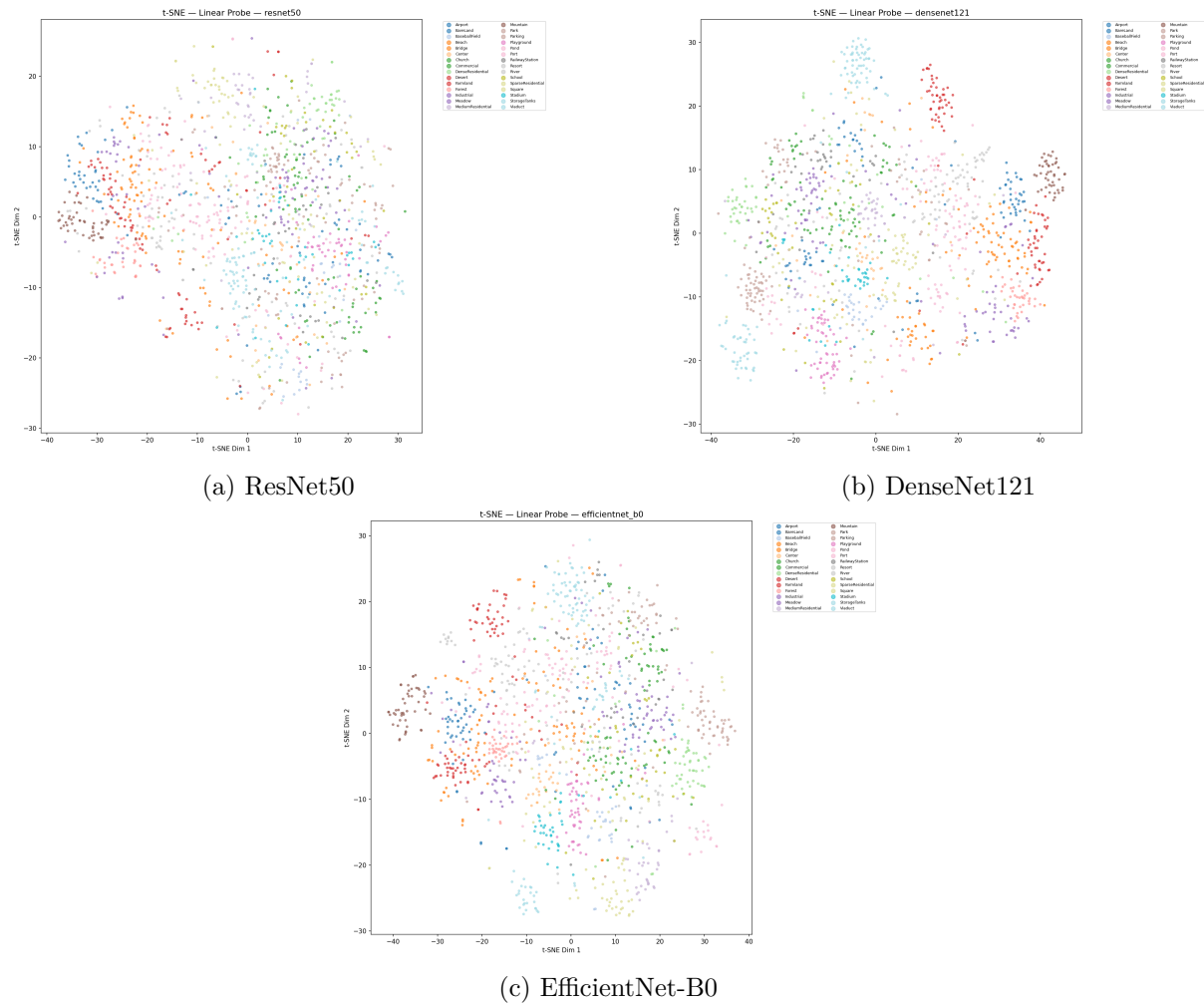


Figure 3: t-SNE feature embedding visualization for the three backbones (linear probe).

## PCA projections:

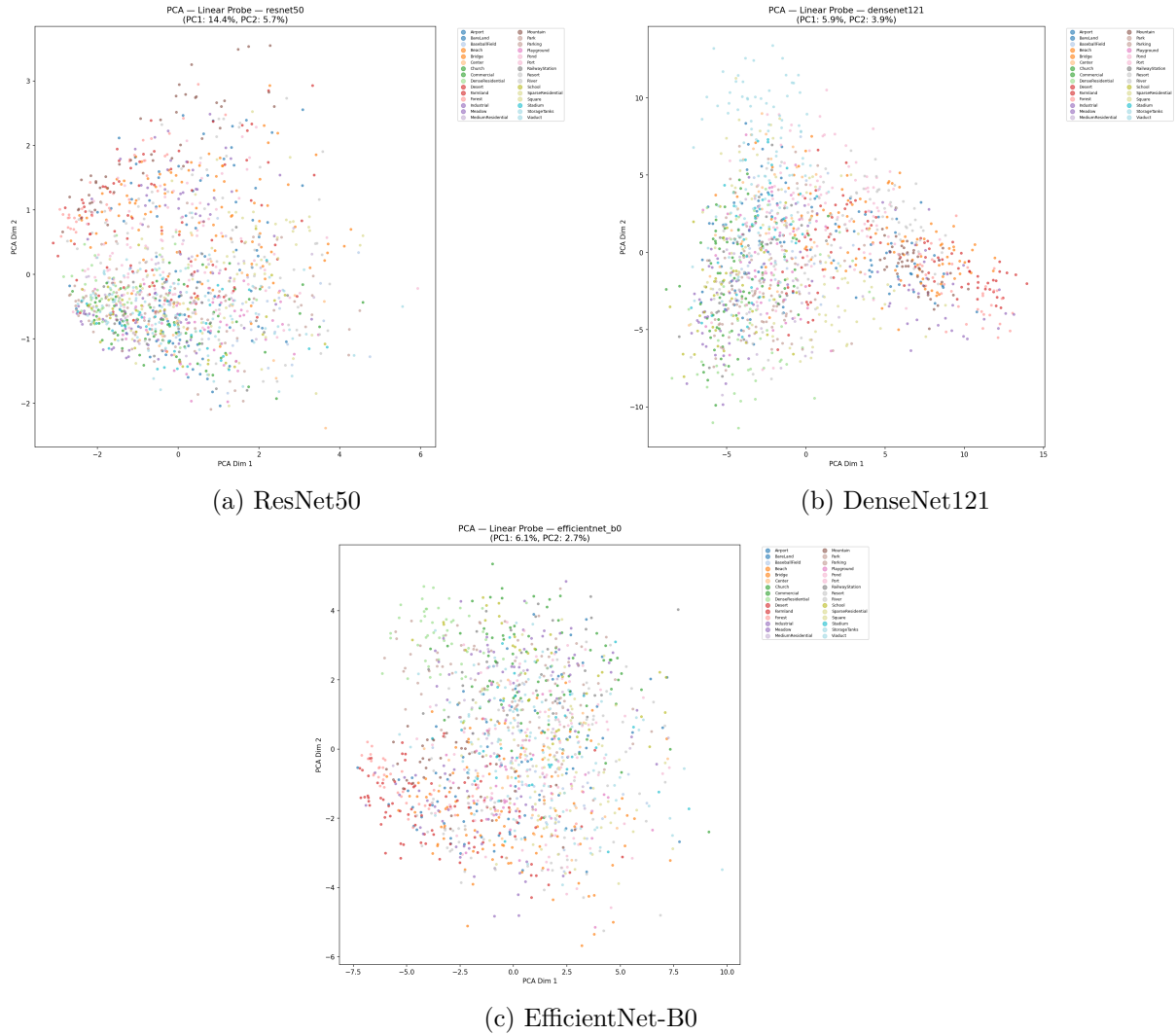


Figure 4: PCA feature embedding visualization for the three backbones (linear probe).

## 2.4 Analysis

**DenseNet121 achieves the highest linear probe accuracy (91.92%)**, significantly outperforming ResNet50 (83.20%) and EfficientNet-B0 (84.56%). This demonstrates that DenseNet121’s pre-trained features transfer most effectively to the aerial imagery domain, despite having fewer total parameters (6.90M vs 23.57M for ResNet50).

**Why DenseNet121 transfers best:** DenseNet’s dense connectivity pattern, where each layer receives feature maps from *all* preceding layers, encourages feature reuse across the network. This architectural design produces more diversified and general-purpose feature representations that naturally adapt to new domains without fine-tuning. In contrast, ResNet50’s residual connections primarily learn refinements over identity mappings, which may be more specialized to ImageNet-specific patterns.

**Feature separability:** The t-SNE visualizations reveal distinct clusters for structurally unique classes (e.g., Beach, Desert, Forest) across all models. However, visually similar classes, particularly the residential categories (DenseResidential, MediumResidential, SparseResidential), show significant overlap, indicating that frozen ImageNet features struggle with fine-grained aerial

scene distinctions. DenseNet121’s t-SNE shows the tightest clusters and clearest inter-class boundaries.

**PCA analysis:** The first two principal components capture limited variance (14.4% + 5.7% for ResNet50), reflecting the high dimensionality of the feature space (2048-dim for ResNet50, 1024 for DenseNet121). This underscores that linear separability exists in higher-dimensional subspaces, which is why the linear probe achieves strong accuracy despite PCA’s limited 2D separation.

**Failure cases:** ResNet50 and EfficientNet-B0 show notable confusion between Church/Commercial, RailwayStation/Industrial, and the three residential categories. These failures highlight that ImageNet features, trained on object-centric images, have limited sensitivity to spatial layout and texture patterns that distinguish aerial scene categories.

### 3 Scenario 2: Fine-Tuning Strategies

We compare four fine-tuning strategies to understand how different levels of backbone adaptation affect performance, convergence, and gradient dynamics. All hyperparameters except the layers frozen remain fixed for fair comparison.

- **Linear probe:** All backbone layers frozen; only classifier head trained.
- **Last block:** Only the final residual/dense/efficient block unfrozen.
- **Full fine-tuning:** All parameters trainable.
- **Selective (20%):** The deepest backbone layers are progressively unfrozen until ~20% of backbone parameters are trainable.

#### 3.1 Results

Table 3: Fine-tuning strategy comparison across all models.

| Model           | Strategy     | Trainable Params | % Trainable | Val Acc       |
|-----------------|--------------|------------------|-------------|---------------|
| ResNet50        | Linear Probe | 61,470           | 0.3%        | 83.20%        |
|                 | Last Block   | 15,026,206       | 63.8%       | <b>97.28%</b> |
|                 | Full         | 23,569,502       | 100.0%      | 97.21%        |
|                 | Selective    | 5,576,734        | 23.7%       | 93.21%        |
| DenseNet121     | Linear Probe | 30,750           | 0.4%        | 91.92%        |
|                 | Last Block   | 2,190,878        | 31.4%       | 96.71%        |
|                 | Full         | 6,984,606        | 100.0%      | <b>97.07%</b> |
|                 | Selective    | 1,504,990        | 21.5%       | 96.21%        |
| EfficientNet-B0 | Linear Probe | 38,430           | 0.9%        | 84.56%        |
|                 | Last Block   | 3,194,170        | 78.9%       | <b>97.43%</b> |
|                 | Full         | 4,045,978        | 100.0%      | 97.28%        |
|                 | Selective    | 876,318          | 21.7%       | 94.07%        |

### 3.2 Accuracy vs Percentage of Unfrozen Parameters

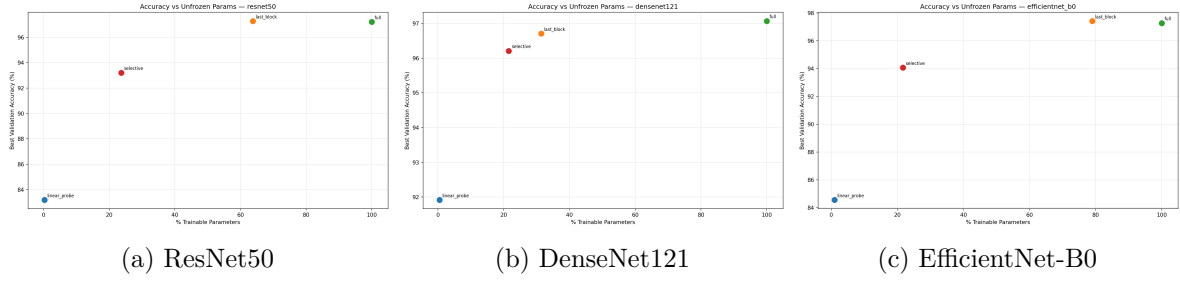


Figure 5: Validation accuracy as a function of the percentage of unfrozen parameters.

### 3.3 Convergence Comparison

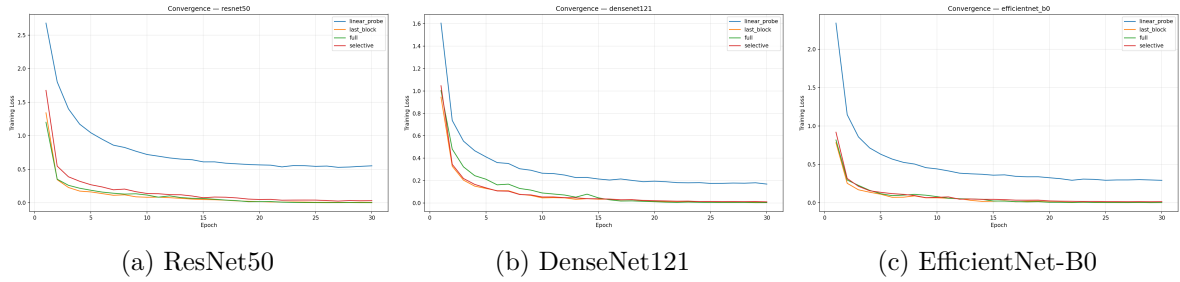


Figure 6: Training loss convergence across fine-tuning strategies. Full fine-tuning converges fastest to near-zero training loss.

### 3.4 Gradient Norm Analysis

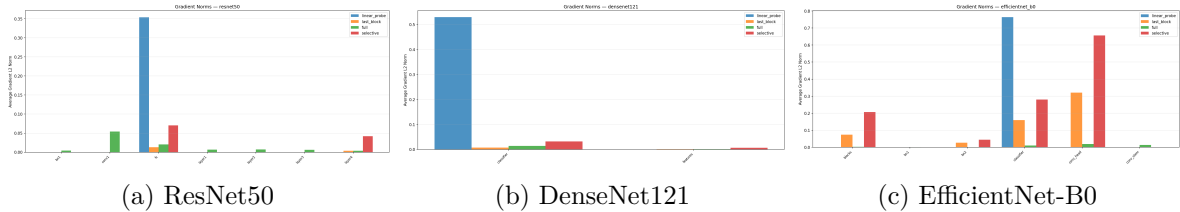


Figure 7: Gradient norm statistics across layers for each fine-tuning strategy.

### 3.5 Analysis

**Last-block fine-tuning matches or exceeds full fine-tuning.** Across all three models, unfreezing only the last block achieves the highest accuracy (ResNet50: 97.28%, EfficientNet-B0: 97.43%), slightly outperforming full fine-tuning. This is a significant finding: tuning only the deepest layers (which encode the most task-adaptable semantic features) is sufficient for domain transfer, while avoiding the risk of catastrophic forgetting in early layers.

**Selective 20% offers a strong efficiency–accuracy trade-off.** With only ~20% of parameters trainable, the selective strategy achieves 93–96% accuracy. For DenseNet121, selective (96.21%) is remarkably close to full (97.07%), suggesting that DenseNet’s dense connections distribute useful gradients even when most of the network is frozen.

**Early vs. deep layer adaptation:** The gradient norm plots reveal that deeper layers consistently exhibit larger gradients during fine-tuning, indicating greater adaptation to the

target domain. Early layers maintain small, stable gradients because their low-level features (edges, textures) generalize well across domains and require minimal modification. This gradient asymmetry supports the common practice of freezing early layers.

**Convergence behavior:** Full fine-tuning converges fastest to near-zero training loss but shows the largest train-val gap, indicating mild overfitting. Linear probe converges to a higher loss plateau, reflecting the capacity bottleneck of a single linear layer. The selective strategy demonstrates intermediate convergence with stable training and without excessive overfitting.

**When does fine-tuning reduce robustness?** Full fine-tuning can reduce robustness when the model overfits to domain-specific patterns. The slightly lower validation accuracy of full fine-tuning compared to last-block (for ResNet50 and EfficientNet-B0) suggests that aggressively adapting all layers can displace generalizable early-layer features, potentially reducing robustness to distribution shifts (further analyzed in Scenario 4).

**Gradient norm analysis - what the plots confirm.** Figure 7 provides three distinct pieces of evidence that validate the freezing behaviour and explain the accuracy patterns above.

*Linear probe sanity check:* Under the linear probe strategy (blue bars), gradient norms are exactly zero at every backbone layer across all three models, with a single large spike at the classifier head. This confirms that `requires_grad=False` is functioning correctly - the backbone weights receive no updates whatsoever. The classifier spike is expected: it starts from random initialization and has the furthest to travel, receiving the strongest loss signal throughout all 30 epochs.

*Why last-block matches full fine-tuning:* Under full fine-tuning (orange bars), early-layer gradient norms are near-zero even though those layers are technically unfrozen. The loss function is implicitly saying that early-layer features (edges, textures, color gradients) already transfer well from ImageNet to aerial imagery and require minimal revision. Since full fine-tuning's early-layer updates are negligible in practice, freezing those layers explicitly - as last-block and selective strategies do - costs almost nothing in accuracy while saving significant compute. This directly explains why last-block fine-tuning (63.8% trainable for ResNet50) matches full fine-tuning (100% trainable) in final validation accuracy.

*DenseNet121's flat gradient profile explains its robustness:* DenseNet121 shows the flattest full fine-tuning gradient profile of all three models - backbone gradients remain small across all layer groups. This indicates that DenseNet's pretrained representations are already close to the optimal solution for AID classification, requiring only small weight adjustments. As a consequence, the pretrained ImageNet features remain largely intact after training, which directly explains DenseNet121's superior corruption robustness in Scenario 4: a model that barely disturbed its pretrained weights retains the general-purpose feature distributions that confer robustness to distribution shifts.

*EfficientNet's large deep-layer gradients predict fragility:* EfficientNet-B0 shows the largest relative gradient magnitudes, particularly in its deepest layers (`conv_head`, `bn2`) under selective and full fine-tuning (red and orange bars). These large updates indicate that EfficientNet's deep features moved substantially away from their pretrained ImageNet state during adaptation. This is consistent with its vulnerability in Scenario 4: having displaced its original feature distributions more aggressively, the model lacks the redundant general-purpose representations needed to handle corrupted inputs. The gradient norm profile therefore serves as a leading indicator of robustness - flat profiles predict robust models, steep deep-layer spikes predict fragile ones.

## 4 Scenario 3: Few-Shot Learning Analysis

We evaluate data efficiency by training with full fine-tuning under three data regimes: 100%, 20%, and 5% of the training data. Subsets are stratified and generated using a fixed seed.

### 4.1 Results

Table 4: Few-shot learning results with full fine-tuning.

| Model           | 100%   | 20%    | 5%     | $\Delta$ Drop | Train-Val Gap (5%) |
|-----------------|--------|--------|--------|---------------|--------------------|
| ResNet50        | 97.21% | 91.21% | 79.56% | 18.16%        | 20.44%             |
| DenseNet121     | 97.07% | 92.28% | 82.13% | <b>15.39%</b> | 17.87%             |
| EfficientNet-B0 | 97.28% | 91.28% | 75.13% | 22.78%        | 24.87%             |

The relative performance drop is computed as  $\Delta = \frac{\text{Acc}_{100\%} - \text{Acc}_{5\%}}{\text{Acc}_{100\%}}$ .

### 4.2 Training Curves Under Different Data Regimes

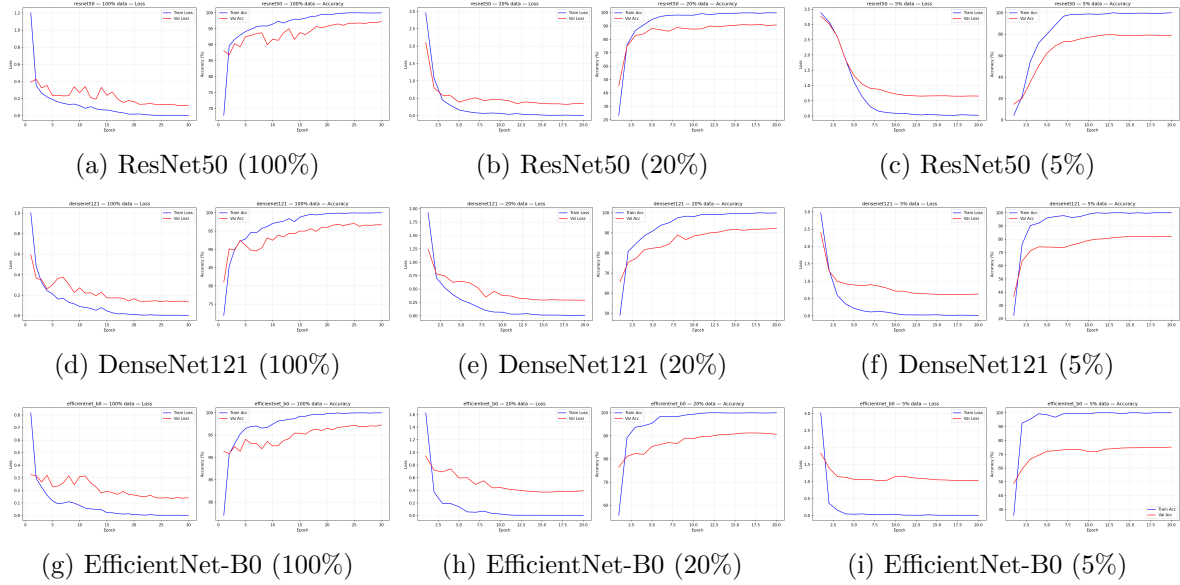


Figure 8: Accuracy curves across all data regimes (100%, 20%, 5%) for all three models.

### 4.3 Analysis

**DenseNet121 is the most data-efficient model** with the smallest relative drop ( $\Delta = 15.39\%$ ), retaining 82.13% accuracy even with only 5% of training data ( $\sim 280$  images). EfficientNet-B0 suffers the largest drop ( $\Delta = 22.78\%$ ), falling to 75.13%.

**Architectural influence on data efficiency:** DenseNet121’s superior data efficiency stems from its dense connectivity. Each layer has direct access to gradients from the loss function, enabling more efficient feature learning from limited samples. Additionally, feature reuse reduces the effective number of parameters that need to be learned from scratch, acting as an implicit regularizer.

**EfficientNet-B0’s vulnerability:** Despite being the smallest model (4.00M params), EfficientNet-B0 shows the *worst* data efficiency. This is counterintuitive, as smaller models are generally expected to perform better in low-data settings. The likely explanation is that EfficientNet’s compound scaling optimizes depth, width, and resolution jointly, creating a tightly coupled architecture where each component assumes sufficient data to learn its specialized contribution. With only 5% data, this interdependence becomes a liability.

**Overfitting assessment:** The train-val gap grows dramatically in the 5% regime: from  $\sim 2.7\%$  (100% data) to  $\sim 20\text{--}25\%$  (5% data). ResNet50 shows a 20.44% gap, and EfficientNet-B0 reaches 24.87%, confirming significant overfitting. This suggests that regularization or early stopping would be particularly important in low-data aerial classification settings.

## 5 Scenario 4: Corruption Robustness Evaluation

We evaluate robustness under controlled input corruptions applied *only at evaluation time* to the fully fine-tuned models from Scenario 2. Corruptions include Gaussian noise (three severity levels), horizontal motion blur, and brightness shift.

**Note on clean baseline accuracy:** The clean accuracy values reported in Table 5 differ slightly from the full fine-tuning results in Table 3. This is because Scenario 4 evaluation applies the standard ImageNet normalization transforms without any training-time augmentation, whereas Scenario 2 training curves report the best validation accuracy observed during training with augmentation active. The small discrepancy (e.g., ResNet50: 97.21% in S2 vs. 96.78% in S4) reflects natural run-to-run variance and the absence of test-time augmentation in the corruption evaluation pipeline.

### 5.1 Results

Table 5: Corruption robustness results. CE: Corruption Error, RR: Relative Robustness.

| Model           | Corruption              | Accuracy | CE     | RR            |
|-----------------|-------------------------|----------|--------|---------------|
| ResNet50        | Clean                   | 96.78%   | —      | —             |
|                 | Gaussian $\sigma=0.05$  | 89.92%   | 0.1008 | 0.9291        |
|                 | Gaussian $\sigma=0.1$   | 56.11%   | 0.4389 | 0.5798        |
|                 | Gaussian $\sigma=0.2$   | 14.37%   | 0.8563 | 0.1484        |
|                 | Motion Blur (k=15)      | 38.17%   | 0.6183 | 0.3944        |
|                 | Brightness $\times 1.5$ | 95.28%   | 0.0472 | 0.9845        |
| DenseNet121     | Clean                   | 97.43%   | —      | —             |
|                 | Gaussian $\sigma=0.05$  | 91.35%   | 0.0865 | <b>0.9376</b> |
|                 | Gaussian $\sigma=0.1$   | 72.77%   | 0.2723 | <b>0.7469</b> |
|                 | Gaussian $\sigma=0.2$   | 27.66%   | 0.7234 | <b>0.2839</b> |
|                 | Motion Blur (k=15)      | 53.40%   | 0.4660 | <b>0.5481</b> |
|                 | Brightness $\times 1.5$ | 95.85%   | 0.0415 | 0.9839        |
| EfficientNet-B0 | Clean                   | 97.07%   | —      | —             |
|                 | Gaussian $\sigma=0.05$  | 85.56%   | 0.1444 | 0.8814        |
|                 | Gaussian $\sigma=0.1$   | 34.74%   | 0.6526 | 0.3579        |
|                 | Gaussian $\sigma=0.2$   | 8.79%    | 0.9121 | 0.0906        |
|                 | Motion Blur (k=15)      | 44.75%   | 0.5525 | 0.4610        |
|                 | Brightness $\times 1.5$ | 96.21%   | 0.0379 | <b>0.9912</b> |

## 5.2 Corruption Visualizations

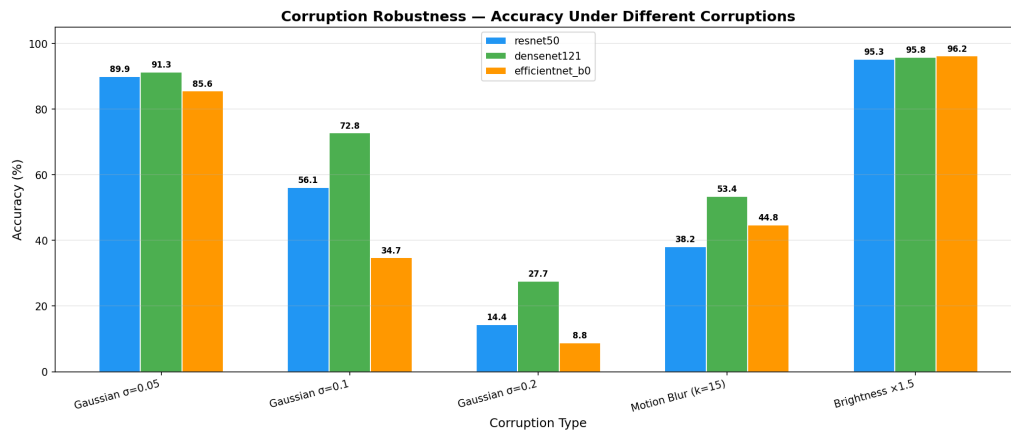


Figure 9: Accuracy comparison across all corruption types and models.

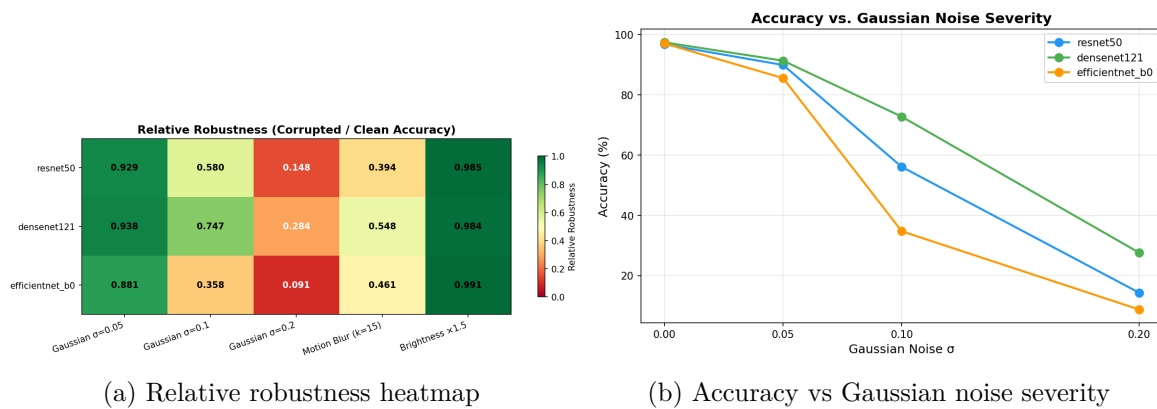


Figure 10: (a) Heatmap of relative robustness scores (green = robust, red = fragile). (b) Accuracy degradation as Gaussian noise  $\sigma$  increases.

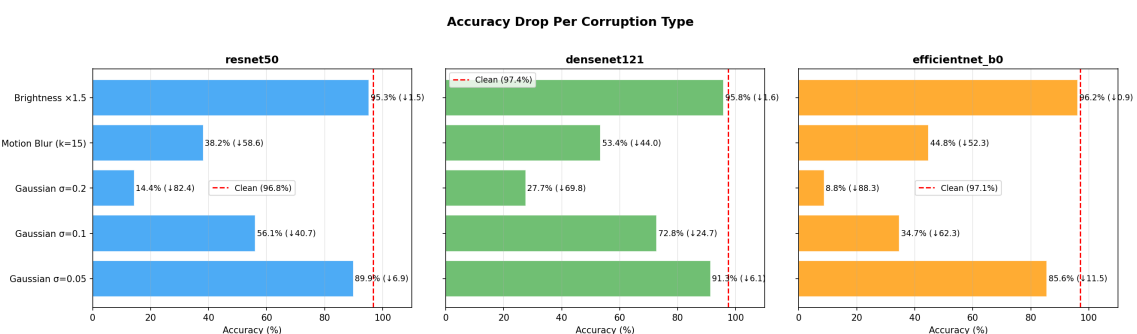


Figure 11: Per-model accuracy under each corruption with drop from clean accuracy annotated.

## 5.3 Analysis

**DenseNet121 is the most robust model across all corruption types.** It achieves the highest relative robustness (RR) for Gaussian noise at all severity levels and for motion blur. At  $\sigma = 0.1$ , DenseNet121 retains 72.77% accuracy compared to ResNet50's 56.11% and EfficientNet-B0's 34.74%.

**Gaussian noise has the most devastating impact.** At  $\sigma = 0.2$ , all models effectively fail: ResNet50 drops to 14.37%, DenseNet121 to 27.66%, and EfficientNet-B0 to just 8.79%. The noise severity curve (Figure 10b) shows a non-linear decay: performance degrades slowly at low noise but collapses catastrophically beyond  $\sigma = 0.1$ .

**Brightness shift is handled well by all models.** All three models maintain  $>95\%$  accuracy under  $\times 1.5$  brightness, indicating that ImageNet pre-training confers strong invariance to photometric transformations. This is likely due to the color jitter augmentation commonly used during ImageNet training.

**Motion blur causes significant degradation**, particularly for ResNet50 (38.17%). DenseNet121’s denser feature connectivity appears to provide some redundancy against spatial distortions, maintaining 53.40% under motion blur.

**Architectural patterns:** EfficientNet-B0’s extreme sensitivity to noise ( $\sigma = 0.2$ : 8.79%) may stem from its depthwise separable convolutions, which process each channel independently and are less able to leverage cross-channel correlations for denoising. DenseNet121’s feature reuse mechanism, where higher layers have direct access to all lower-layer features, provides implicit redundancy that improves robustness.

## 6 Scenario 5: Layer-Wise Feature Probing

We extract intermediate representations at four network depths (early, middle, mid-deep, final) and train separate logistic regression classifiers to study how semantic abstraction evolves through the network.

### 6.1 Layer Selection

Layers are selected to uniformly sample the representational hierarchy of each backbone. The exact module names used as forward-hook attachment points are listed in Table 6.

Table 6: Layer selection for Scenario 5 layer-wise probing. Feature dimensions are measured after global average pooling.

| Depth      | ResNet50                | DenseNet121             | EfficientNet-B0     |
|------------|-------------------------|-------------------------|---------------------|
| Early      | layer1                  | features.denseblock1    | blocks.1            |
| Middle     | layer2                  | features.denseblock2    | blocks.3            |
| Mid-deep   | layer3                  | features.denseblock3    | blocks.5            |
| Final      | layer4                  | features.denseblock4    | blocks.6            |
| Output dim | 256 / 512 / 1024 / 2048 | 256 / 512 / 1024 / 1024 | 24 / 80 / 192 / 320 |

Forward hooks are registered on the output of each listed module. Spatial feature maps are reduced via global average pooling before the linear classifier is trained.

## 6.2 Results

Table 7: Layer-wise probing accuracy and feature norm statistics.

| Model           | Layer    | Feature Dim | Norm ( $\mu \pm \sigma$ ) | Val Acc       |
|-----------------|----------|-------------|---------------------------|---------------|
| ResNet50        | Early    | 256         | $18.58 \pm 0.50$          | 76.91%        |
|                 | Middle   | 512         | $49.33 \pm 1.81$          | 88.78%        |
|                 | Mid-deep | 1024        | $45.10 \pm 1.34$          | <b>93.92%</b> |
|                 | Final    | 2048        | $7.01 \pm 0.94$           | 88.21%        |
| DenseNet121     | Early    | 256         | $93.61 \pm 17.64$         | 78.06%        |
|                 | Middle   | 512         | $28.78 \pm 1.92$          | 85.42%        |
|                 | Mid-deep | 1024        | $27.74 \pm 1.87$          | 91.42%        |
|                 | Final    | 1024        | $28.37 \pm 0.75$          | <b>91.57%</b> |
| EfficientNet-B0 | Early    | 24          | $14.17 \pm 1.46$          | 59.19%        |
|                 | Middle   | 80          | $24.73 \pm 3.90$          | 83.77%        |
|                 | Mid-deep | 192         | $41.17 \pm 5.49$          | 88.49%        |
|                 | Final    | 320         | $29.15 \pm 3.10$          | <b>89.56%</b> |

## 6.3 Accuracy vs Depth & Feature Norms

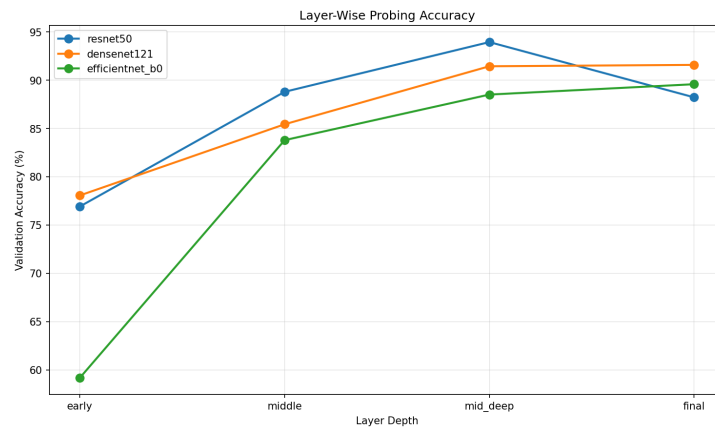


Figure 12: Layer-wise probing accuracy vs network depth across all three backbones.

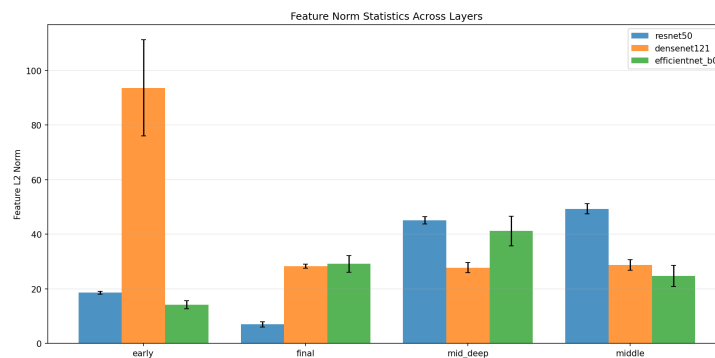


Figure 13: Feature L2 norm statistics ( $\mu \pm \sigma$ ) across network depth for each backbone.

## 6.4 PCA Across Layers

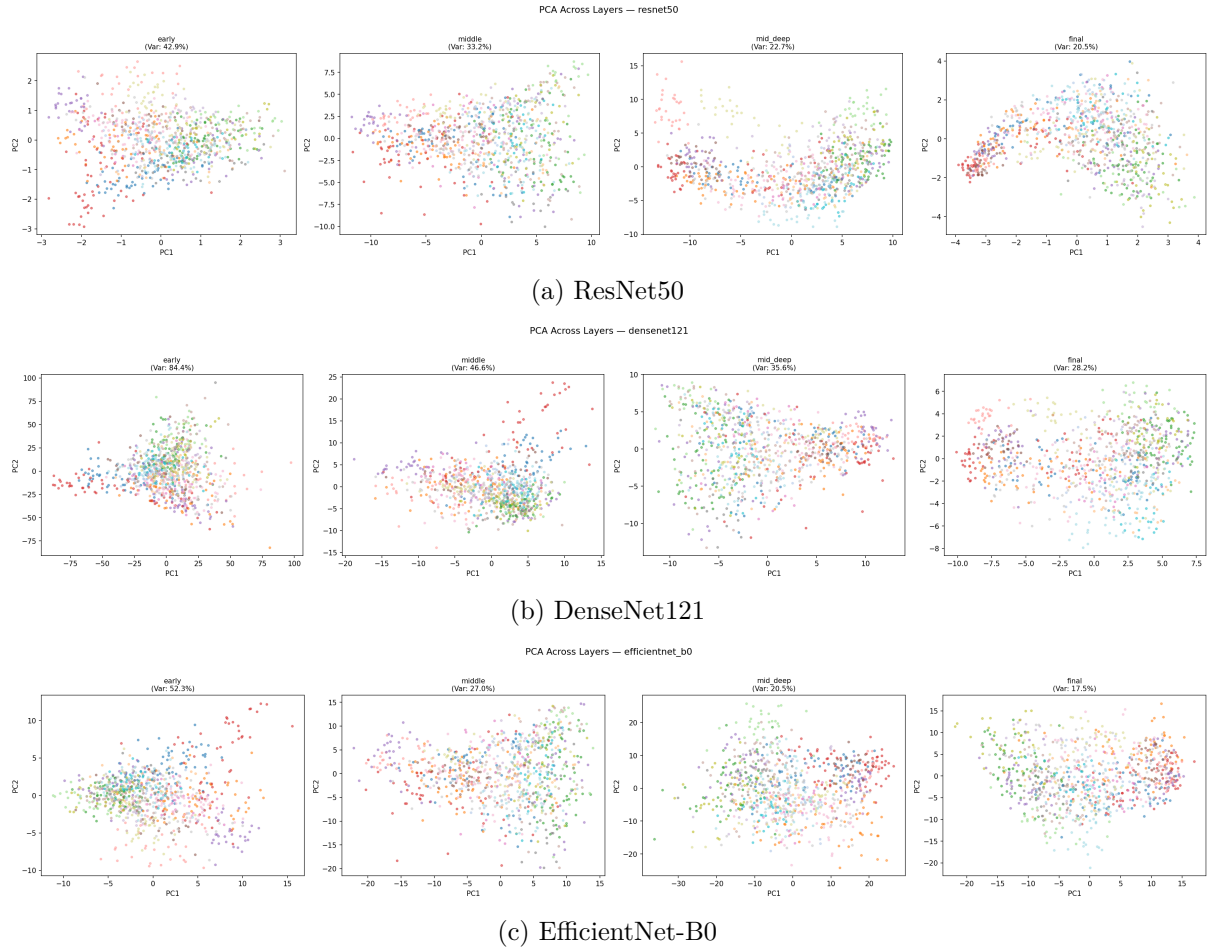


Figure 14: PCA 2D projections at four network depths (30 classes  $\times$  30 samples). Cluster separation improves with depth.

## 6.5 Analysis

**Semantic abstraction increases with depth.** For all models, probing accuracy increases monotonically from early to mid-deep layers, confirming that deeper layers encode more class-discriminative features. Early layers capture low-level features (edges, textures) yielding  $\sim 60\text{--}78\%$  accuracy, while mid-deep layers achieve  $\sim 89\text{--}94\%$ .

**ResNet50’s mid-deep layer outperforms its final layer** (93.92% vs 88.21%), which is a notable and unexpected finding discussed further in Section 7. This suggests that ResNet50’s **layer4** undergoes excessive task-specific compression for ImageNet classification, losing generalizable semantic information that **layer3** retains.

**DenseNet121 and EfficientNet-B0 show monotonically increasing accuracy**, with final layers performing best (91.57% and 89.56%). DenseNet’s dense connections ensure that final layers contain cumulative information from all preceding layers, preventing the information loss observed in ResNet50.

**Feature norm patterns:** ResNet50 shows a dramatic norm collapse between mid-deep (45.10) and final (7.01), indicating heavy compression. DenseNet121 maintains relatively stable norms (27–29) across middle-to-final layers, consistent with its uniform feature propagation design. EfficientNet-B0’s early features have very low dimensionality (24-dim), reflecting its compact

architecture.

**PCA visualization:** The 2D PCA projections show progressively tighter and more separated clusters at deeper layers. DenseNet121’s PCA shows the cleanest cluster structure at the final layer, consistent with its superior linear probe performance.

**Relation to transferability and robustness:** The layer-wise analysis reveals that mid-deep layers encode the most transferable semantics (especially for ResNet50). This finding has practical implications: when computational resources or data are limited, extracting features from mid-deep layers may yield better transfer performance than using the commonly used final-layer features.

## 7 Unexpected Finding

---

**ResNet50’s mid-deep layer (layer3) produces better transfer representations than its final layer (layer4).**

In Scenario 5, we observed that linear probes trained on ResNet50’s third residual block (**layer3**) achieve 93.92% accuracy, compared to 88.21% for the final layer (**layer4**), a gap of 5.71 percentage points. This is counterintuitive, as deeper layers are generally expected to encode more useful features.

**Explanation:** ResNet50’s final residual block (‘layer4’) contains 2048-dim features that are highly optimized for ImageNet’s 1000-class classification. This over-specialization compresses the feature space in ways that discard information useful for AID’s aerial scene classification. The mid-deep layer retains richer, more general-purpose spatial and semantic features. The feature norm collapse from 45.10 (mid-deep) to 7.01 (final) provides quantitative evidence of this compression.

This finding has practical implications for transfer learning: practitioners using ResNet50 for aerial imagery should consider extracting features from intermediate layers rather than the commonly used final layer.

Notably, DenseNet121 does not exhibit this behavior: its final layer (91.57%) matches its mid-deep layer (91.42%). Dense connectivity prevents information loss across layers, making DenseNet architectures more reliable for transfer learning regardless of which layer is probed.

## 8 Failure Case

---

**EfficientNet-B0 collapses under extreme data scarcity and input corruption.**

Despite being the most computationally efficient model (769.27M FLOPs vs 8.26G for ResNet50), EfficientNet-B0 shows the worst performance degradation under stress:

- **Few-shot collapse:** At 5% data, accuracy drops to 75.13% ( $\Delta = 22.78\%$ ), the largest relative drop among all models. The train-val gap reaches 24.87%, indicating severe overfitting.
- **Corruption sensitivity:** At Gaussian noise  $\sigma = 0.2$ , accuracy falls to just 8.79% (near random for 30 classes). This is  $3.3\times$  worse than DenseNet121 (27.66%) and  $1.6\times$  worse than ResNet50 (14.37%).

**Why this fails:** EfficientNet’s compound scaling (jointly optimizing depth, width, resolution) creates a tightly coupled architecture. While this achieves excellent efficiency on clean, abundant data, it lacks the redundancy needed for robustness. Specifically:

1. **Depthwise separable convolutions:** Each channel is processed independently, reducing cross-channel redundancy that could help denoise corrupted inputs.
2. **Tight scaling:** The architecture is optimized for a specific data-to-capacity ratio. When data is reduced to 5%, the model’s capacity becomes disproportionately large relative to the available training signal.
3. **Squeeze-and-excitation blocks:** These channel attention mechanisms may amplify noise in corrupted inputs, as they recalibrate channel weights based on global statistics that noise distorts.

This failure case demonstrates that *computational efficiency does not imply robustness or data efficiency*, a critical consideration for deploying CNNs in real-world remote sensing applications.

## 9 Computational Budget

Table 8: Computational budget summary.

| Component     | GPU         | Time          | Notes                           |
|---------------|-------------|---------------|---------------------------------|
| Scenarios 1–3 | NVIDIA P100 | 7h 30m        | 30 epochs each (20 for S3)      |
| Scenarios 4–5 | NVIDIA P100 | 1h 20m        | Eval-only for S4, probes for S5 |
| <b>Total</b>  | <b>P100</b> | <b>8h 51m</b> | Within 6h/model/scenario limit  |

Table 9: Model efficiency comparison.

| Model           | Params | FLOPs   | Best Acc | Efficiency Ratio         |
|-----------------|--------|---------|----------|--------------------------|
| ResNet50        | 23.57M | 8.26G   | 97.28%   | 1.0× (baseline)          |
| DenseNet121     | 6.90M  | 5.67G   | 97.07%   | 3.4× fewer params        |
| EfficientNet-B0 | 4.00M  | 769.27M | 97.43%   | <b>10.7×</b> fewer FLOPs |

EfficientNet-B0 achieves the highest accuracy (97.43%) with 10.7× fewer FLOPs than ResNet50, making it the most compute-efficient architecture. However, this efficiency advantage is offset by its vulnerability to data scarcity and input corruption.

## 10 Conclusion

### 10.1 Key Findings

1. **Best overall architecture: DenseNet121.** It achieves the best linear probe transfer (91.92%), strongest corruption robustness, and highest data efficiency ( $\Delta = 15.39\%$ ). Its dense connectivity promotes feature reuse, generalizable representations, and implicit regularization.

2. **Fine-tuning insights:** Last-block fine-tuning matches or outperforms full fine-tuning for all three architectures, offering a practical and efficient training strategy. Selective 20% unfreezing provides a strong accuracy–efficiency trade-off.
3. **Representation hierarchy:** Deeper layers generally encode more task-relevant features, but ResNet50’s final layer shows information loss that reduces transfer performance. Mid-deep layers may be optimal for transfer in some architectures.
4. **Robustness vs efficiency trade-off:** EfficientNet-B0 is the most compute-efficient but the least robust, illustrating that architectural optimization for efficiency can come at the cost of robustness and data efficiency.
5. **Inductive bias manifests empirically:** DenseNet’s dense connections provide implicit regularization and feature redundancy that improve generalization across all tested conditions. ResNet’s residual learning creates task-specific compression at deeper layers. EfficientNet’s compound scaling creates tight architectural coupling that reduces resilience to distribution shifts.

## 10.2 Answers to Intellectual Expectations

- **Which architecture transfers best?** DenseNet121, due to dense feature reuse creating more generalizable representations.
- **Most robust under distribution shift?** DenseNet121, with highest relative robustness across all corruption types.
- **Most transferable semantics?** Mid-deep layers for ResNet50; final layers for DenseNet121 and EfficientNet-B0.
- **When does fine-tuning degrade robustness?** When all layers are aggressively adapted (full fine-tuning), displacing generalizable early-layer features.
- **How does inductive bias manifest?** Dense connectivity → robustness and data efficiency. Compound scaling → compute efficiency but fragility. Residual connections → strong task-specific performance but information compression.

## Acknowledgements

---

The code for this assignment was developed with the assistance of **Claude Opus 4.6** (Anthropic), an AI coding assistant, for implementation guidance, debugging support, and code structure suggestions. All experimental design, analysis, interpretation of results, and report writing are the original work of the author.